BASIC NETWORKING COMMANDS OF WINDOWS AND LINUX

Command 1:

Window: ipconfig

Linux: ip addr

purpose: Show your IP address and networking details.

Command 2:

Window: Ipconfig /all

Linux: ip address show

purpose: Shows Complete network configurat

Command 3:

Window: ping google.com

Linux: ping google.com

purpose: Checks if another device or website is reachable.

Example output:

Wireless lan Adapter:

Ipv6 Address . . . : 2406:7400:ce:4faf:1acc:2ffc

Ipv4 Address . . . : 192.168.88.9

Subnet Mask . . . : 255.255.255.0

output:

Windows IP Configuration

Host name . . . : DESKTOP-ICNBRGCW

Primary DNS Suffix . . . .

Node TYPE . . . : Mixed.

IP Routing Enabled : No

output:

Pinging google.com [142.250.77.110] with
32 bytes of data.

Reply from 142.250.77.140 bytes:
32 time = 5ms TTL = 119.

Reply from 142.250.77.140 bytes:
32 time = 6ms TTL = 119.

Approximate round trip times
in milli = Sec:

Minimum = 5ms, max = 8ms, Avg = 6ms

Command 4:

Window: Tracert google.com

Linux: Tracert google.com

Purpose: Shows the path packets take to a destination.

Command 5:

Window: nslookup google.com.

Linux: nslookup google.com.

Purpose: Finds IP address of a Domain.

Command 6:

Window: netstat -an

Linux: netstat -tuln

Purpose: Display Active network Connections

Output:

Tracing route to google.com [142.250.77.110]
over a maximum of 30 hops:

1	4ms	1ms	2ms	172.16.8.1
2	24ms	3ms	12ms	137.97.163.129
3	*	*	*	Request Time out
4	17ms	5ms	5ms	172.16.12.122

Trace Complete.

output:

Server: unknown
Address: 172.16.8.1
Non-authoritative answer:
Name: google.com
Address: 2404:6800:4007:805::2002
142.250.67.46

Output

Active Connections

proto	Local Address	Foreign Address	State
TCP	0.0.0.0.135	0.0.0.0.0	LISTENING
TCP	0.0.0.0.445	0.0.0.00	LISTENING
TCP	172.10.11.150.50756	4.213.25.240.243	LISTENING
TCP	172.10.11.150.61909	184.28.109.178	TIME_WAIT
TCP	172.16.11.150.61918	74.226.80.120	CLOSE_WAIT
TCP			

Command 7:

Window : arp -a

Linux : arp -a

Purpose : Shows IP- to- MAC address mapping.

Command 8:

Window : hostname

Linux : hostname

Purpose : Display Computer Name.

Command 9:

Window : getmac

Purpose : Show MAC Address.

Command 10:

~~Linux~~ : route -n or ip route

Purpose : Shows routing table.

(80)

RESULT:

The implementation of Basic network commands in windows and Linux has been verified.

Bally 7/26

output:

Interface : 172.16.11.150 --- 0x8

Internet Address	physical Address
172.16.8.1	7C-59-1C-CF-be-45
255.255.255.255	FF-FF-FF-FF-FF

Interface : 192.168.66.1 -- 0xb

Internet Address	physical Address	TYPE
192.168.56.255	FF-FF-FF-FF-FF	Static
224.0.0.22	01-00-5e-00-16	Static

output:

Window → DESKTOP-IK1BRGCV

Linux → prathap-PC

output:

physical Address	Transport Name
E8-80-88-85-A5-D7	\Device\
DC-46-28-1F-52-ED	TCPIP_{4C2895EF-AA5

output:

Default via 192.168.1.1 dev eth0

192.168.1.0/24 dev eth0 proto kernel

192.168.1.100.

OF NETWORK CABLES

AIM:
To Study and Understand different types of network Cable. To Make your own Ethernet Cross over Cable / through Cable.

a) Understand Different types of network Cable.

1) Unshielded Twisted pair (UTP)
2) Shielded Twisted pair (STP) Cable.
3) Coaxial Cable.
4) Fibre optic Cable.

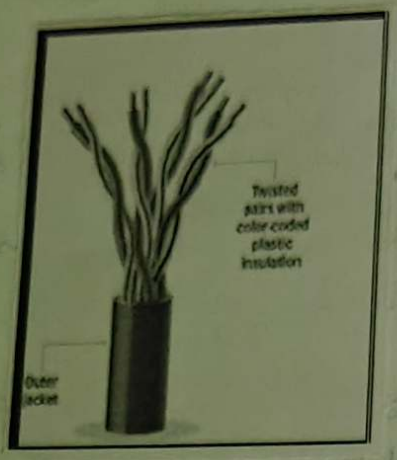
Cable Type	Category	Max Data Transmission	Advantages / Dis-advantages	Ap
UTP	Category 3	10 bps	Advantage → Cheaper	10 B Ethe
	Category 5	upto 100 Mbps	→ Easy to install Smaller Diameter.	Fas
	Category 5e	1 Gbps	Dis-ad.	Eth
			More prone to (EMI) Electro magnetic field.	
STP	Category 6 6a.	10gbps	Ad Shielded Faster than UTP.	Gig 10
SSTP	Category 7	10gbps.	Dis-ad → Expensive → Greater installation effort	Gi 10

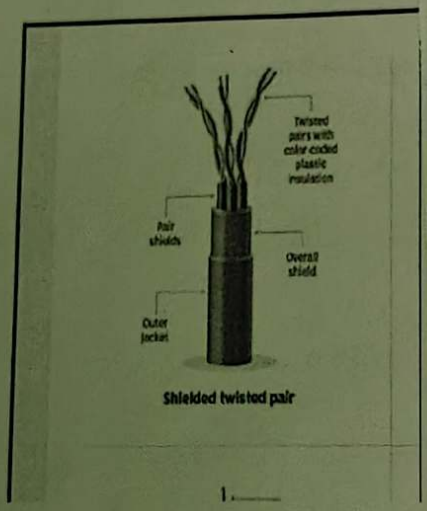
Cable type	Category	Maximum data Transmission	Advantage Dis-Adv	Application / Use
Coaxial Cable	RG-6 RG-59 RG-11	10 - 100Mbps	-> High bandwidth -> Low loss bandwidth. -> versatile Dis-Adv: -> limited Distance -> Size is bulky	Speed of Signal is 500m Television. network High speed internet Connections.
Fibre optics Cable	Single mode Multi mode	100Gbps	Advantages -> High Speed -> High Bandwidth. -> High Security Dis/Adv -> Expensive -> Requires skilled installers	Maximum Distance of fibre optics Cable around 100 meters

-> Make your own Ethernet Cross-over Cable / Straight Cable / Straight Cable.

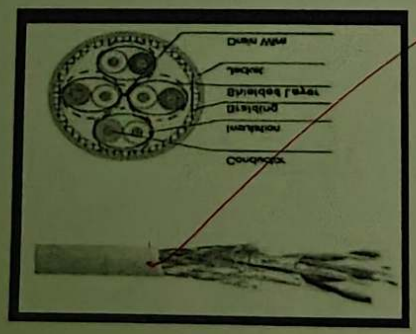
-> Two RJ45 plugs

-> optional two plug Shields.





Shielded twisted pair



Step 1:
To start Construction of Device. Begin by threading shields onto Cable.

Step 2:
Next, Strip Approximately 1.5cm of Cable Shielding from both ends.

Step 3:
After, you will need to untangle the wires. There should be Four "twisted pairs".

Step 4:
Once the order is Correct, Bunch them together in a Line, and if there to Do so, facing away from you and have the Gold pins facing toward you as Shown.

Step 5:
Next, Push the Cable right in. The notch at the end of plug needs to just over Cable Shielding.

95

Bijil 14/8/20

RESULT:
Thus, different types of network Cables were studied Successfully.

UNDERSTANDING CISCO PACKET TRACKER ENVIRONMENT

AIM:
To understand Cisco packet Tracer environment and design simple Computer network using interface & networking Components.

STEPS:

-> open CISCO packet Tracer application
-> observe user interface, including MenuBar, Main ToolBar
-> from Network Component Box, select required network Devices such as. PCs, Hub, Switches.
-> Drag and Drop the Devices into work space.
-> Connect Devices using Copper Straight through Cables.
-> Verify that all links are active (green LED)
-> observe packet transmission in Simulation Mode.
-> Save Network topology.

RESULT:
The Cisco packet Tracer environment was Successfully Explored.

HUB

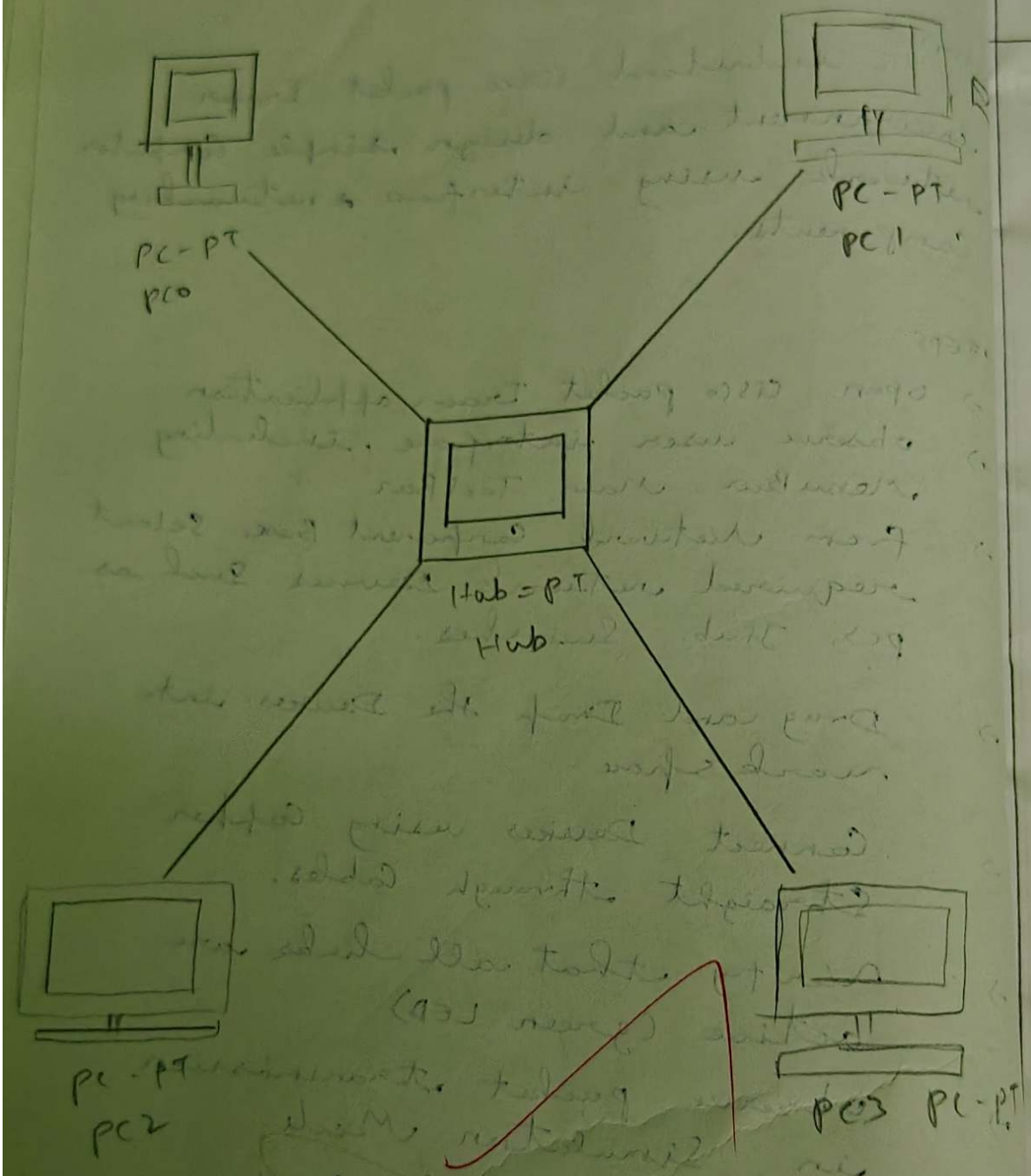
EX.NO 3.A

AIM:
To study behaviour of a Hub observing how it forwards data packets using Cisco packet Tracer.

STEPS:
→ open Cisco packet Tracer.
→ Drag and place to hub using Copper Straight - Through Cables.
→ Configure ip addresses for all pcs in same networks
→ Select Add Simple PDU from Common Tools Bar.
→ Send a packet from one pc to another pc.
→ Switch Simulation Mode and observe how hub forwards the packet.



RESULT:
The hub broadcasts received packet to all Connected Devices.


PC-PT
PC0
PC-PT
PC1
Hub = P.T.
Hub
PC-PT
PC2
PC3
PC-PT

STUDY OF WIRESHARK

AIM:
To study the Experiments on Packet Capture tool: Wireshark.

CAPTURING AND ANALYSING PACKETS USING WIRESHARK TOOL:

To filter, Capture, view, packets in Wireshark Tool.

PROCEDURE:

→ Select Local Area Connection in Wireshark.
→ Go to Capture option
→ Select Stop Capture Automatically. after 100 packets.
→ Save the packets.

OUTPUT

1. Create a Filter to Display only Tcp / UDP packets, inspect the packets and provide flow Graph.

2. Create a Filter to Display only ARP packets and inspect the packets.

PROCEDURE:

→ Go to Capture option.

→ Select Stop Capture automatically after 100 packets.

→ Then click start Capture.

→ Search ARP packets in Search Bar.

→ Save the packets.

3. Create Filter Display only DNS packets and provide the flow Graph.

procedure:

→ Go to Capture option.

→ Select Stop Capture automatically after 100 packets.

→ Then click start Capture.

→ Search DNS packets in Search Bar.

→ To see flow Graph click Statistics. Flow Graph.

4. Create a Filter to Display only HTTP packets and inspect the packets procedure.

→ Select Local Area Connection in Wireshark.

→ Go to Capture option.

→ Select Stop Capture Automatically After 100 packets.

→ Then click Start Capture.

→ Search HTTP packets in Search Bar.

5. Create a Filter to Display only DHCP packets and inspect the packets. procedure.

→ Select Local Area Connection in Wireshark.

→ Go to Capture Automatically after 100 packets.

→ Then click Start Capture.

→ Save the packets.

6. Create a Filter to Display only DHCP packets and inspect the packets.

PROCEDURE:

→ Select Local Area Connection in Wireshark.

→ Go to Capture option.

→ Select Stop Capture Automatically after 100 packets.

→ Then click Start Capture.

→ Save the packets.

80

RESULT: Therefore the output wireshark is Verified and Completed.

16/8/20

SETUP AND CONFIGURE LAN

AIM:

To Setup and Configure a LAN (Local Area Network) Using a Switch and Ethernet Cables in your Lab.

What is LAN?

Local Area Network (LAN) refers to Network that Connects Devices within a Limited Area. Such as office, building, School or home. It enables users to share resources, including data, printers, and Internet Access. LAN Connects Devices to promote Collaboration and transfer information between users.

How to Setup a LAN:

Step 1: plan and Design an Appropriate network topology taking into Account network requirements and equipment location.

Step 2: You can take 4 Computers, A Switch with 8, 16 or 24 ports which is Sufficient for networks of these Sizes and 4 Ethernet Cables.

Step 4:

Assign Ip Address to your pc's

i) Log on to client Computer as Administor

ii) click Network and internet Connection.

Step 5:

Select a folder → goto properties → click sharing

80

RESULT: Therefore the output of Setuping and Configuring LAN is Verified and Completed.

14/8/18

Ex: 6 : Error detection and Correction using Hamming Code.

AIM:

To write a program to implement error detection and Correction using Hamming Code.

ALGORITHM:

1. Start the program
2. Read the number.
3. forming the input data stream.
   a. Compute $p1 = d_1$ XOR d_2 XOR d_4.
      $p2 = d_1$ XOR d_3 XOR d_4.
      $p3 = d_2$ XOR d_3 XOR d_4.
4. Compute error position = C4 C2 C1
5. If Error position = 0, no error has occured.
6. Extract the bit at "Error position".
7. Repeat steps 3 to 8 for rael n blocks
8. Stop the program.

Block I :

Enter 4 data bits
(Space Seperated eg. 1011): 1011

Original data bits: 1011

Encoded Hamming Code (7 Bits): 0110011

Enter Bit position to introduce
error (1-7, 0 = no error)

Transmitted Code (with Error
included at pos 3): 0100011

Recieved code: 0100011

Corrected Code: 0110011

Recovered data bits: 1011

PROGRAM:

```
def encode(d):
    """d: list of 4 bits [d1, d2, d3, d4]
    h = [0] * 7          h[0] = h[2] ^ h[4] ^ h[6]
    h[2] = d[0]          h[1] = h[2] ^ h[5] ^ h[6]
    h[4] = d[1]          h[3] = h[4] ^ h[5] ^ h[6].
    h[5] = d[2]
    if error_pos = 0
        print(f"Enter detected at bit position:
    {error_pos} -> Correcting.")
    else:
        print("No error detected")
    return error_pos
for b in range(1, n+1):
    print(f"\n-- Block {b} ---")
    d = list(map(int, input("Enter 4 data Bit").
    h = encode(d)
    print(f" original data bits")
    print(f" Encoded Hamming code {to_str}")
    detect_and_Correct(h)
    print(f" Corrected Code: {to_str(h)}")
if __name__ == "__main__":
    main()
```

(80)

B.dll 4/8/26

RESULT: Therefore the error detection and correction using Hamming Code is verified and completed.

Block 2:

Enter 4 data bits (Space Separated, eg

original data bits : 0 1 1 0

Encoded Hamming Code (7 Bits) : 1 1 0

Received Code : 1 1 0 0 1 0 0

Checking for errors.

Corrected Code : 1 1 0 0 1 1 0

Recovered data Bits : 0 1 1 0

Ex: 7 Flow Control at Data Link layer using sliding window protocal.

AIM:

To write a program to implement flow Control at data link layer using the sliding window protocal, and to simulate the flow of frames from one node to another.

ALGORITHM:

1. Start the program
2. Read total number of frames to be sent (N).
3. Repeat while base < N:
   a. Determine Current Sender window.
   b. The Reciever discards the last frame and Subsequent.
4. Repeat Step 4 until all N frames have been transmitted. acknowledged.
5. Stop the program.

Enter total no of frame to send : 8
Enter Sender's window size : 4
Enter frame no to Simulate as Lost

Sender Window [frame 0, frame 1, AI

node A → Sending frame 0
node A → Sending frame 1
node A → Sending frame 2
node A → Sending frame 0, Sending
node B → Recieved frame 0, Sending AC AL
node B → Recieved frame 2, Sending

Sender window: [frame 4 - frame 7]
node A → Sending frame 4.
node A → Sending frame 5
node A → Sending frame 6
node A → Sending frame 7.

Sender window: [frame 4 - frame 7]
node A → Sending frame 5
node A → Sending frame 6.
node B → Recieved frame 4, Sending AC
node B → Recieved frame 5, Sending A
node B → Recieved frame 7, Sending AC

PROGRAM:

```
def Simulate (total_frames, window_size):
    base = 0
    next_seq = 0

while base < total_frames:
    window_end = min( base+window_size-1,
    total_frames-1)
if (last_name is Name and base_list_fram
(next Seq:
print (f"Node B -> Frame {last_frame}.
for f in range (base, next_Seq):
    Print (f"Node B -> Recieved frame{f}:
    Sending ACK {p}")
    base = next_Seq:
def main():
    total_frame = int (input("Enter
        total number to Send: "))

if __name__ == "__main__":
main()
Simulate (total_frame, window_size,
lost_frame)
```

(80)

RESULT:

Therefore the program to implement flow control is Verified by Simulating flow two holds.

Bell 8/10/25

IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER.

AIM:

To implement classless Ip Subnetting using a 127 Subnet mask and Configure routers, Switches and ACS in Cisco packet tracer. and to Verify Connectivity between Subnets using the Ping Command.

PROCEDURE:

→ afar Cisco packet tracer and Create new topology.

→ Add 2 routers, 2 Switches and pcs.

→ Connect the devices using Copper

→ Divide 192.168.1.0124 into 127 Subnets.

→ Configure router interface with Ip address and Subnet mask.

→ Configure Switches ports as Access ports.

→ Test Connectivity used the Ping Command.

Configuring the Devices:

Now that we have added our devices and Connected them, We can start Configuring them we will start by router, Right click on the router and Select "CLI", this will open the Command-Line interface for the router. In CLS enter following Commands

* enable
* interface terminal
* ip address {Ip address} {Subnet mask}
* no shutDown.
* exit.
* interface fast Ethernet 0/1
* exit.

Next, we will Configure the Switch, Right click on the Switch and Select "CLI"; In the CLI. enter following Commands.

enable
Configure terminal
interface Fast Ethernet 0/1
Switch port mode access.
exit

These Commands Configure the Switch to operate in access mode on its two ports. which are connected to the two PCs.

Commands - enable
Configure Terminal
interface Gigabit Ethernet 0/0
ip address {Ip address} {Subnet mask} no
Shut Down
exit.

RESULT:
The network was Successfully Configured using classless IP Subnetting. Connectivity between router and Verified using Ping Command.

10A Internetworking with Router in CISCO packet tracer.

AIM:

To design and Configure a Simple Internetworking using a router and two PCs in CISCO packet tracer and Verify Connectivity between the PCs.

PROCEDURE:

* Open CISCO packet tracer & Create a new workspace.
* Add one router (router1) and two PCs (PC0 and PC1)
* Connect:
  - PC0 Fast Ethernet 0 → Router 1
  - PC1 Fast Ethernet 0 → Router 1.
- Ip addresses : 192.168.10.2
  Subnet Mask : 255.255.255.0
  Default Gateway : 192.168.10.1
- wait until the links become
- Assign Appropriate Ip address and Subnet masks to all PCs.

- Enable router Interfaces using no shutdown and Configure routing in Simulator mode
- Test Connectivity between PCs using Ping Command.

RESULT: The routers were successfully Configured in CISCO packet tracer. and Connectivity was Verified using ping.

Design and Configure an internetwork. using wireless router DHCP Server and internet cloud.

AIM:

To Design and Configure an internetwork. using a wireless router, DHCP server and Internet cloud in CISCO packet tracer.

PROCEDURE:

→ open Cisco packet tracer.

→ Add a wireless Router, PC/Laptop and internet / WAN Setting according to the network.

→ Connect the wireless router's internet / WAN Setting according to network.

→ Configure the wireless pc/laptop to wireless router using the Configured SSID.

→ Set the pc / laptop Ip Configuration to DHCP So that it automatically recieves an Ip address.

→ Verify the pc/ Laptop.

→ Use the Simulation mode, to observe the packets.

RESULT:

The internetwork was successfully designed and configured using a wireless router, DHCP Server, and internet cloud in Cisco Packet tracker

IMPLEMENT ECHO CLIENT SERVER USING TCP/UDP SOCKETS.

AIM:

To implement an echo client Server application using TCP/UDP Socket programming in Java and demonstrate end to end Communication at the transport layer.

PROCEDURE:

1. Create a Server program and client program using JAVA Socket programming.
2. For TCP, Create Server Socket and Server and Socket at the client.
3. Assign a Suitable port number for Communication.
4. Start the Server and make it wait for client Connection.
5. Start the client and establish Connection with Server.
6. The client Sends a message to the Server.
7. The Server recieves the message and Sends Same message back to the client.
8. The client recieves and displays the echoed message.

9. For UDP Create Datagramsocket objects at th client and Server.

10. The client recieves and displays echo message and Verify Successful Comm between client and Server.

N map to discover Live Hosts.

AIM:

To Discover live hosts in a network using N map by performing ARP Scan, ICMP Scan, TCP Ping Scan and UDP Ping Scan on the Try Hack Me platform.

PROCEDURE:

→ Log in to the Try Hack Me platform.

→ Open the Nmap room and start the target machine.

Start the Attack box and open the terminal.

Enter the target machine is Ip Address.

Perform ARP, ICMP, TCP and UDP Scans using Nmap Commands.

Observe and Record Live hosts Discovered.

RESULT:

Thus, live hosts in the network were discovered Successfully using Nmap. with ARP, ICMP, TCP and UDP Scanning techniques.

Implement a chat program using Socket Programming.

AIM:

To implement a Simple chat client Server application using TCP Sockets for Communication between clients

ALGORITHM:

-> Start the TCP Server and Create a Serversocket.

-> Accept the client Connection

-> Create a thread to Handle each client.

-> Read message sent by the client.

-> close the Connection.

CODE:

```
java.net.*;
class chatclient

    private static final unit Server_port = 5000;
    Public static Void main (String [] args

    socket = new Socket (Server
           = APPEAR SERVER_ PORT);

    Buffer Reader = new Buffer Reader
      new input Stream Reader (Socket.get.
```

```
PrintWriter out = new PrintWriter(
socket.getOutputStream(), autoflush.
);
BufferedReader consol = new BufferedReader(
    new InputStreamReader(System.in));
};
String message;
while ((message = consol.readline()) !=
  if (message.equals("bye")) {
  System.out.println("client closed").
```

RESULT:

Thus, The client chat-socket application was successfully implemented using TCP/UDP Socket.